Classical AI Reasoning Agent

AI Assignment 2026 — Public Benchmark Experiments

1. Objective

The system predicts whether a query is supported by a knowledge base containing facts and rules. Its pipeline is: input context and query → internal logical representation → classical reasoning → Boolean prediction. The implementation uses task-specific paths for RuleTaker-style natural-language facts and rules, CounterBench causal counterfactual questions, and FOLIO first-order logic examples.

2. Representation

Horn-clause representation. For fact-and-rule tasks, facts and rules are represented as literals. A literal contains a predicate, arguments, and a negation flag. A rule has a body of premises and a head, corresponding to a form such as $A(x) \wedge B(x) \rightarrow C(x)$. Variables are unified with constants when rules are matched against known facts.

First-order logic (FOL). The FOLIO path uses an expression tree with atoms, negation, binary connectives, and quantifiers. The code includes formula tokenization and parsing, substitution, and a satisfiability/entailment pipeline involving Skolemization, subject-sorted Herbrand grounding, Tseitin CNF, and DPLL/resolution-related routines.

CounterBench causal representation. The counterfactual path extracts structural causal equations, observations, and intervention conditions from the supplied text. It follows the three-stage pattern abduction → action/intervention → prediction and returns a yes/no result.

3. Reasoning algorithms

The agent combines several classical techniques: (i) **forward chaining** to derive consequences from known facts and applicable rules; (ii) **backward chaining** to work backwards from a query to subgoals; (iii) **unification** to match variables in rules with terms in facts; and (iv) a configurable search procedure for backward chaining, set to A* in the notebook. The FOL path additionally uses logical normalization and a SAT-style search procedure. Reasoning traces are available for sample queries to show intermediate derivations.

4. Public-dataset experiments

The following values are copied from the executed output saved in the submitted notebook. Inference times depend on the runtime environment and should be treated as measurements from that run.

Dataset split	Queries	Correct	Accuracy	Total time	ms/query
RuleTaker-Train	1,000	983	98.30%	0.2664 s	0.266
RuleTaker-Test	250	244	97.60%	0.0668 s	0.267
RuleTaker-OOD-Test	375	373	99.47%	0.0915 s	0.244
CounterBench-Alpha	1,000	960	96.00%	0.2131 s	0.213
FOLIO-Validation	203	169	83.25%	14.2130 s	70.015
FOLIO-Train	1,001	835	83.42%	87.5245 s	87.437

Reported aggregate	Correct / total	Accuracy	Total inference time
Held-out public splits	1,746 / 1,828	95.51%	14.5843 s
All evaluated public splits	3,564 / 3,829	93.08%	102.3752 s

Evaluation note. In the notebook, FOLIO labels are converted to a Boolean target by treating only the label *True* as positive; *False* and *Uncertain* both become False for the reported accuracy calculation. Therefore, the FOLIO figures reflect this binary mapping, not a three-class FOLIO evaluation. The held-out aggregate also combines different benchmark types and should not be interpreted as a guaranteed private-test score.

5. Example reasoning traces

Example A — Forward and backward chaining

Knowledge base: Bob is young; Bob is kind; if someone is young, they are smart; if someone is smart and kind, they are successful. Query: “Bob is successful.”

```
1. Initial facts: kind(bob), young(bob)
2. Apply young(x) → smart(x), deriving smart(bob).
3. Apply smart(x) ∧ kind(x) → successful(x), deriving successful(bob).
4. Forward chaining proves the query in two rule firings. The backward-chaining trace also resolves
successful(bob) into subgoals smart(bob) and kind(bob), then proves them from the rules and facts.
```

Example B — CounterBench counterfactual

The saved trace records structural equations including Ziklo → Blaf, Blaf ∧ Ziklo → Trune, Trune → ¬Vork, and Vork → Lumbo. The query intervenes with do(Ziklo = False). The agent then evaluates the structural equations and reports Lumbo = True, producing the answer “yes.” This illustrates the implementation's intervention-and-prediction path; it is an example trace, not a proof that every causal pattern is handled correctly.

Example C — FOLIO query

For the validation example “Tom is an Ocellated wild turkey,” the notebook trace reports a parsed FOL knowledge base/conclusion and a three-way logical status of True, which is converted to a Boolean True prediction by the agent.

6. Limitations and conclusion

The experiments show strong results on the evaluated RuleTaker splits and CounterBench-Alpha, while FOLIO is both less accurate and substantially slower in the recorded run. The implementation uses hand-written natural-language patterns and logical parsing, so its coverage depends on the forms those parsers recognize; the results should not be generalized to arbitrary language or unseen logical forms without further testing. The notebook output reports 1,828 predictions across the selected held-out public splits and states that it generated a CSV with columns `id,label`.

The assignment requires a general-purpose classical reasoning system and evaluates accuracy and total inference time. The notebook demonstrates the intended combination of logical representations and classical inference methods, but public benchmark performance is not a guarantee of private-test performance.